



Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

ACTIVIDAD IV - Sumador Nibble y 8 bits

Documento elaborado por: Aaron Ricardo Segura Gutiérrez

Arquitectura de Computadoras
Profesor:
Jorge Ernesto Lopez Arce Delgado
SECCIÓN D05 2026A

Implementación de sumadores en Verilog

El presente trabajo se realiza con el fin de explicar el desarrollo de un sumador de 4 y de 8 bits, desde la base que es un Half Adder (sumador de 1 bit), Full Adder (sumador de 2 bits), instancias de ambas, así como instancias de el sumador de 4 bits.

Introducción:

¿Cómo se relaciona con Verilog?

La implementación de la Lógica Digital se realiza mediante la programación de FPGAs en los sistemas digitales, todas las operaciones matemáticas se realizan utilizando números binarios, es decir, combinaciones de 0 y 1. Una de las operaciones más importantes en cualquier sistema digital es la suma, ya que muchas otras operaciones como la resta, la multiplicación y el funcionamiento interno de los procesadores dependen de ella. Por esta razón, comprender cómo funciona un sumador digital es fundamental en el estudio de la arquitectura de computadoras y el diseño lógico.

Cuando se realiza una suma en sistema binario, se siguen reglas similares a las del sistema decimal, pero solo utilizando dos valores. En algunos casos, al sumar dos bits se genera un acarreo (carry), el cual debe sumarse en la siguiente posición. Este proceso es muy importante, ya que si no se maneja correctamente el acarreo, el resultado puede ser incorrecto. En sumadores de varios bits, el acarreo se va propagando de una posición a otra.

En esta práctica se desarrollaron dos sumadores: uno de 4 bits y otro de 8 bits. El sumador de 4 bits permite sumar números desde 0 hasta 15, mientras que el de 8 bits permite sumar números desde 0 hasta 255. Ambos reciben dos números binarios y un bit adicional llamado acarreo de entrada. Como resultado, generan la suma correspondiente, y en el caso del sumador de 8 bits, también se obtiene un acarreo de salida cuando el resultado es mayor al valor máximo que se puede representar.

Objetivos:

Esta actividad busca que se entienda que la operación básica más importante es la suma, así como saber los diferentes que tipos de sumadores existen sus características, diferencias, ventajas y desventajas. Del mismo modo se debe saber que tipo de sumador se implementa en una GPU y en un CPU.

Desde el punto de vista de Verilog la intención de esta actividad es que se entienda y se ponga en práctica la instancia de módulos previamente hechos, y la implementación de módulos con entradas y salidas usando buses o vectores.

Se realizarán dos sumas, una de un ancho de palabra de 4 bits, esto es que el sumador tendrá dos operando de entrada de 4 bits c/u, y otra de ancho de palabra de 8 bits, esto es que cada operando tendrá 8 bits y el resultado 9.

**Obtenido del PDF en Classroom*

Desarrollo:

El sumador estructural se construyó utilizando módulos más pequeños llamados Full Adder (FA). En este tipo de diseño se conectan manualmente las etapas de suma, simulando cómo estaría construido físicamente el circuito.

Primero se diseñó el módulo Full Adder, el cual suma tres bits (A, B y Cin) y produce una suma y un acarreo.

Después, para construir el sumador de 4 bits, se instanciaron cuatro Full Adders conectados en cascada

Para el sumador de 8 bits, se instanciaron ocho Full Adders siguiendo el mismo principio.

El diseño estructural es más detallado porque muestra cómo se propaga el acarreo entre cada bit. Esto permite comprender mejor el funcionamiento interno del circuito.

El sumador comportamental se diseñó utilizando el operador aritmético + de Verilog. Este tipo de diseño describe directamente la operación que se quiere realizar, sin especificar cómo está construido internamente el circuito.

Para el sumador de 4 bits se utilizó la siguiente estructura:

```
assign S = A + B + c_in;
```

El tamaño de la salida se definió con un bit adicional por el acarreo de salida.

Para el sumador de 8 bits se utilizó:

```
assign {C_out, S} = A + B + C_in;
```

Aquí se separa el acarreo de salida (C_out) del resultado (S). Este diseño es más claro porque permite identificar fácilmente cuándo ocurre overflow.

El sumador comportamental es más sencillo de escribir, más compacto y más fácil de entender. Sin embargo, no muestra explícitamente cómo se realiza la suma internamente.

TESTBENCH

Los testbench fueron diseñados para verificar el correcto funcionamiento de los sumadores comportamentales de 4 y 8 bits. Su función principal es aplicar diferentes combinaciones de entradas y observar si las salidas generadas coinciden con los resultados esperados.

Un testbench no tiene entradas ni salidas físicas, ya que solo se utiliza para simulación. Dentro de él se declaran señales tipo **reg** para las entradas y tipo **wire** para las salidas del módulo que se desea probar.

EVIDENCIAS

```
1#
1 module bit8_c(
2     input  [7:0] A,
3     input  [7:0] B,
4     input  C_in,
5     output [7:0] S,
6     output C_out
7 );
8
9 assign {C_out, S} = A + B + C_in;
10
11 endmodule
12
13
```

```
C:/Users/aaron/Documents/Arquitectura de Computadoras/Simulador/Simulador/
1#
1 module bit4_c(
2     input  [3:0] A,
3     input  [3:0] B,
4     input  c_in,
5     output [4:0] S
6 );
7
8 assign S = A + B + c_in;
9
10 endmodule
11
12
```

```

module bit4(
    input  [3:0] A,
    input  [3:0] B,
    input  i_c,
    output [3:0] C
);

wire c1, c2, c3, c4;

FA f1 (.A(A[0]), .B(B[0]), .i_c(i_c), .S(C[0]), .o_c(c1));
FA f2 (.A(A[1]), .B(B[1]), .i_c(c1), .S(C[1]), .o_c(c2));
FA f3 (.A(A[2]), .B(B[2]), .i_c(c2), .S(C[2]), .o_c(c3));
FA f4 (.A(A[3]), .B(B[3]), .i_c(c3), .S(C[3]), .o_c(c4));

endmodule

```

```

module bit8 (
    input  [7:0] A,
    input  [7:0] B,
    input  c_in,
    output [8:0] S
);

wire c1;

bit4_c b1( .A(A[3:0]), .B(B[3:0]), .c_in(c_in), .S({c1, S[3:0]}));

bit4_c b2(.A(A[7:4]), .B(B[7:4]), .c_in(c1), .S(S[8:4]));

endmodule

```

Conclusión:

En esta práctica pudimos ver cómo funciona un sumador digital tanto de 4 bits como de 8 bits usando dos formas diferentes de programación en Verilog. El sumador comportamental fue el más fácil de hacer porque solo usamos el operador `+`, lo que hace que el código sea más corto y simple de entender.

En cambio, el sumador hecho con instancias fue un poco más largo y detallado, ya que se conectan varios módulos pequeños para formar uno más grande. Aunque es más complejo, ayuda a entender mejor cómo funciona realmente el circuito por dentro, especialmente cómo se pasa el acarreo de un bit a otro.

Los testbench fueron importantes porque nos permitieron comprobar que todo funcionara correctamente. Al repetir las pruebas varias veces, se confirmó que las sumas y los acarros se calculaban bien.

En general, esta práctica ayudó a entender que un mismo circuito puede hacerse de distintas maneras, y que cada forma tiene sus ventajas dependiendo de lo que se quiera lograr.

Referencias:

GeeksforGeeks. (s.f.). Verilog Tutorial. Recuperado de: <https://www.geeksforgeeks.org>

W3Schools. (s.f.). Programming Tutorials. Recuperado de: <https://www.w3schools.com>

YouTube. (s.f.). Tutoriales de Verilog y ModelSim. Recuperado de:

<https://www.youtube.com>

Electronics Tutorials. (s.f.). Digital Logic Tutorials. Recuperado de:

<https://www.electronics-tutorials.ws>